

AltoMoney: Phaze Solo Review Security Review

Cantina Solo review by:
Phaze, Lead Security Researcher

June 18, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	<code>maxWithdraw()</code> and <code>maxRedeem()</code> may overstate liquidity for paused markets . . .	4
3.2	Gas Optimization	5
3.2.1	Supply to markets proceeds with stale interest data when interest accrual is blocked	5
3.3	Informational	6
3.3.1	Dust share positions can prevent immediate cleanup-only full unwind during reallocation	6
3.3.2	Vault accounting assumes it is not configured as a borrow market fee recipient	6
3.3.3	Re-enabling a removed market double-counts stale vault supply and inflates accounting	7
3.3.4	<code>erc4626Mint()</code> and <code>erc4626Withdraw()</code> do not sweep leftover tokens unlike USM counterparts	9
3.3.5	Current guardian can indefinitely block its own replacement by revoking pending guardian transitions	10
3.3.6	Lost asset recovery mechanism and its accounting implications should be documented more thoroughly	11

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Alto is a decentralized credit protocol built around DUSD, focused keeping risk contained by isolating markets, using a single stablecoin and partial liquidations.

From Mar 18th to Mar 23rd the security researchers conducted a review of [v1](#) on commit hash [b1d504fc](#). A total of **8** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	1	1	0
Gas Optimizations	1	1	0
Informational	6	6	0
Total	8	8	0

2.1 Scope

The security review had the following components in scope for [v1](#) on commit hash [b1d504fc](#):

```
contracts
├── lending
│   ├── libraries
│   │   ├── AltoBalancesLib.sol
│   │   ├── MarketLib.sol
│   │   └── PendingLib.sol
│   └── vault
│       └── AltoVault.sol
└── utils
    └── AltoAdapter.sol
```

3 Findings

3.1 Low Risk

3.1.1 `maxWithdraw()` and `maxRedeem()` may overstate liquidity for paused markets

Severity: Low Risk

Context: `AltoVault.sol#L857-L884`

Summary: The vault's `maxWithdraw()` and `maxRedeem()` helpers may report more withdrawable liquidity than is actually accessible when a queued market is paused. The preview path still counts the vault's balance in that market, while the execution path cannot withdraw from it because `removeSupply()` is blocked by the market pause, which can cause withdrawals based on the advertised maximum to revert with `NotEnoughLiquidity()`.

Description: The vault estimates withdrawable liquidity by iterating the withdraw queue in `_simulateWithdrawFromMarkets()` and subtracting the amount that appears withdrawable from each market based on `expectedMarketBalances()` and `_withdrawable()`. For a paused market, `expectedMarketBalances()` only skips virtual interest accrual; it still returns the market's current supply and borrow balances, so the simulation continues to count that market's apparent liquidity.

The execution path differs materially. `_withdrawFromMarkets()` attempts to call `removeSupply()` on each queued market inside a `try/catch` block and silently skips markets that revert. Because `removeSupply()` is gated by `whenNotPaused`, a paused market is deterministically unavailable during execution even though its liquidity was included by the preview path. If the remaining unpaused markets cannot satisfy the requested amount, the withdrawal reverts with `NotEnoughLiquidity()` despite `maxWithdraw()` or `maxRedeem()` indicating that the amount should be available.

This is not a solvency issue and does not create a direct path to loss of funds. However, it is a real mismatch between the vault's advertised ERC-4626 withdrawal limits and the amount that can actually be withdrawn under the current state.

Impact Explanation: Users and integrators may rely on overstated `maxWithdraw()` and `maxRedeem()` values and submit withdrawals that revert unexpectedly. This may reduce composability for integrations that depend on accurate ERC-4626 limit helpers and may degrade user experience during paused-market incidents.

Likelihood Explanation: The issue is straightforward to trigger whenever the vault has material funds in a queued market that is currently paused and the remaining markets cannot cover the requested amount. Market pauses are permissioned and are expected to be relatively uncommon, which keeps the overall likelihood lower than fully permissionless issues.

Recommendation: Consider making `_simulateWithdrawFromMarkets()` conservative with respect to current execution constraints by skipping markets that are paused before counting their liquidity. One approach could be:

```
function _simulateWithdrawFromMarkets(uint256 assets) internal view returns (uint256) {
    for (uint256 i; i < withdrawQueue.length; ++i) {
        address market = withdrawQueue[i];
+
+         // Do not count liquidity that cannot be accessed while the market is paused.
+         if (IPausable(market).paused(PAUSE_TYPE)) continue;

        uint128 supplyShares = market.userSupplyShares(address(this));
        (uint256 totalSupplyAssets, uint256 totalSupplyShares, uint256
        ↪ totalBorrowAssets,) =
            market.expectedMarketBalances();
```

Alto Money: Fixed in [PR 492](#).

Phaze: Fix verified.

3.2 Gas Optimization

3.2.1 Supply to markets proceeds with stale interest data when interest accrual is blocked

Severity: Gas Optimization

Context: [AltoVault.sol#L808-L825](#)

Description: The `_supplyToMarkets()` function in `AltoVault` iterates over the supply queue and, for each market, first attempts to accrue interest before calculating how much to supply. The `accrueInterest()` call is wrapped in a try/catch block so that a paused or reverting market does not block the entire deposit flow:

```
try IMarket(market).accrueInterest() {} catch {}

(uint128 totalSupplyAssets, uint128 totalSupplyShares) = market.totalSupply();
uint128 supplyShares = market.userSupplyShares(address(this));
uint256 supplyAssets = supplyShares.convertToAssetsUp(totalSupplyAssets,
    ↪ totalSupplyShares);

uint256 toSupply = UtilsLib.min(supplyCap.zeroFloorSub(supplyAssets), assets);
```

When `accrueInterest()` reverts and is caught, execution continues with stale `totalSupplyAssets` and `totalSupplyShares` values that do not reflect pending interest. This causes `supplyAssets` to be underestimated, potentially inflating `toSupply` beyond what the supply cap should allow.

The same pattern exists on the withdrawal side in `_withdrawFromMarket()`, where `accrueInterest()` is also wrapped in a try/catch before reading market state. In this case, stale data would cause the vault to undervalue its position in the market, potentially withdrawing fewer assets than it is entitled to. From a risk perspective, this direction is less concerning for the vault, as undervaluing its own holdings is conservative. However, the inconsistency remains.

In practice, both `addSupply()` and `removeSupply()` internally call `_accrueInterest()` before updating any balances, so the scenario where the external `accrueInterest()` fails but the internal one succeeds is unlikely, as both follow the same code path. Additionally, these functions share the `whenNotPaused` modifier, meaning a paused market would cause both calls to revert. This limits the practical impact to an edge case where the external call fails for a transient reason that resolves by the time the supply operation executes.

Nevertheless, if `accrueInterest()` fails, proceeding with stale data to calculate supply amounts and then attempting the supply or withdraw operation is unnecessary work at best. Skipping the market entirely when interest accrual fails would be both safer and more gas-efficient.

Recommendation: Consider adding an early `continue` or `return` when `accrueInterest()` fails, skipping the market for that cycle. This avoids calculations based on stale data and saves gas on the subsequent operation, which would likely also fail.

In `_supplyToMarkets()`:

```
- try IMarket(market).accrueInterest() {} catch {}
+ try IMarket(market).accrueInterest() {} catch { continue; }
```

In `_withdrawFromMarket()`, a similar early return could be applied depending on whether the vault should allow withdrawals to proceed without current interest data.

Alto Money: Fixed in [PR 497](#).

Phaze: Fix verified.

3.3 Informational

3.3.1 Dust share positions can prevent immediate cleanup-only full unwind during reallocation

Severity: Informational

Context: [AltoVault.sol#L401-L452](#)

Description: `reallocate()` treats `allocation.assets == 0` as a request to fully unwind a market position, but that path is only reached if the vault's current position first produces a nonzero `withdrawn` amount. When the vault holds nonzero `supplyShares` whose rounded-down asset value is zero, `_accruedSupplyBalance()` reports `supplyAssets == 0`, so `reallocate()` skips the withdrawal branch entirely and leaves the dust shares in place.

In practice, this means a cleanup-oriented `reallocate()` call may fail to clear a market whose remaining position is only dust at the share level. A subsequent `updateWithdrawQueue()` call still sees nonzero `userSupplyShares(address(this))`, so the market cannot be removed through the immediate empty-market path even though the remaining position has no rounded-down asset value. The protocol's documented forced-removal flow remains available, so this is an operational edge case rather than a meaningful lock or fund-risk issue.

Because the remaining position is worth zero assets under the vault's own rounding model, the impact is limited to allocator convenience and cleanup semantics. The behavior may still be worth addressing if the intended meaning of `allocation.assets == 0` is "remove all shares" rather than "remove all nonzero-valued assets."

Recommendation: Consider handling the `allocation.assets == 0` case before checking whether the rounded-down asset amount is nonzero, and allow `reallocate()` to call `removeSupply(0, supplyShares, ...)` whenever the vault still holds shares. This would make the cleanup path consistent for dust-only positions without changing any material economic outcome.

Alto Money: Fixed in [PR 493](#).

Phaze: [PR 493](#) fixes this by explicitly handling dust positions in `reallocate()`. The original code only entered the withdraw path when `withdrawn > 0`, so dust positions with `supplyAssets == 0` but `supplyShares != 0` were skipped. The patched code uses `if (withdrawn > 0 || (allocation.assets == 0 && supplyShares != 0))` and, for `allocation.assets == 0`, withdraws by passing `shares = supplyShares` to `removeSupply()`.

3.3.2 Vault accounting assumes it is not configured as a borrow market fee recipient

Severity: Informational

Context: [AltoVault.sol#L922-L952](#)

Description: The vault values its market positions using `expectedSupplyAssets(address(this))`, but the helper used for that calculation is explicitly documented as incorrect for a market's `feeRecipient` because it does not include virtually accrued fee shares in the recipient's own share balance. In borrow markets, virtual accrual increases total supply shares to reflect pending fee shares, but the corresponding user-level share balance is only updated after real on-chain accrual.

If a borrow market used by the vault is configured with the vault itself as `feeRecipient`, the vault may temporarily undercount its own position in that market during view-based accounting. This can make the vault's reported asset base and share pricing slightly inconsistent until the market is accrued on-chain. In practice, this appears to be an unusual configuration choice rather than an expected deployment pattern, so the issue is better viewed as an implicit configuration assumption that is not enforced in code.

Recommendation: Consider making this assumption explicit by rejecting markets whose `feeRecipient` is the vault when enabling them, or by documenting that such a configuration is unsupported. If supporting that setup is desirable, the accounting path could instead be adjusted to include virtually accrued fee shares when valuing the fee recipient's position.

Alto Money: Fixed in [PR 494](#).

Phaze: This edge case is now documented.

3.3.3 Re-enabling a removed market double-counts stale vault supply and inflates accounting

Severity: Informational

Context: [AltoVault.sol#L770-L796](#)

Summary: When a market is force-removed from the vault's withdraw queue while residual supply shares remain and is later re-enabled, the vault can count the same underlying position twice. The residual position is first preserved in vault accounting after removal, and then added again as live assets when the market is re-enabled, inflating `lastTotalAssets` and distorting share pricing.

Description: The vault enables markets through `_setCap()`, which adds `market.expectedSupplyAssets(address(this))` to `lastTotalAssets` whenever a market transitions from disabled to enabled. Market removal occurs in `updateWithdrawQueue()`, where a market can be omitted from the new withdraw queue after the curator has submitted removal and the timelock has elapsed, even if the vault still holds nonzero supply shares there.

When this forced-removal path is used, the residual position remains on the market's books but is removed from the vault's queue and configuration. `updateWithdrawQueue()` deletes `config[market]` without adjusting `lastTotalAssets`, so the removed position is not explicitly derecognized from accounting at the moment of removal. Once the market is no longer in `withdrawQueue`, the vault's asset aggregation no longer reads that live position directly, and the accounting model preserves the missing amount through its existing loss-tracking path.

If the same market is later re-enabled, `_setCap()` treats it as a fresh market and unconditionally adds `market.expectedSupplyAssets(address(this))` to `lastTotalAssets` again. At that point, the same residual position is represented twice in accounting: once through the preserved accounting base left behind by removal, and once through the newly re-added live market balance.

This creates ghost assets in the vault's accounting. ERC-4626 share conversions then use an inflated asset base, causing new depositors to receive fewer shares than they should while existing shareholders can redeem against overstated value.

Impact Explanation: The inflated accounting can redistribute value between vault participants. New depositors may mint materially fewer shares than warranted, while pre-existing holders may redeem against the inflated share price and extract disproportionate value. The effect persists until the stale position is fully resolved.

Likelihood Explanation: Triggering the issue requires a specific sequence of privileged actions: the market must be force-removed with residual supply still present, and then later re-enabled before that residual position is cleared. These are operationally uncommon steps, but they are permitted by the current code paths and do not require any invariant break elsewhere.

Proof of Concept:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

import "../helpers/IntegrationTest.sol";
import {console2} from "forge-std/console2.sol";

contract ReenableRemovedMarketGhostAssetsPoC is IntegrationTest {
    using AltoBalancesLib for address;

    IERC4626 internal erc4626Vault;

    uint256 internal constant INITIAL_DEPOSIT = 100 ether;
```

```

function setUp() public override {
    super.setUp();

    erc4626Vault = IERC4626(address(vault));
}

function test_poc_reenableRemovedMarketDoubleCountsResidualSupply() public {
    loanToken.setBalance(SUPPLIER, INITIAL_DEPOSIT);

    vm.prank(SUPPLIER);
    uint256 attackerShares = erc4626Vault.deposit(INITIAL_DEPOSIT, SUPPLIER);

    assertEq(attackerShares, INITIAL_DEPOSIT, "initial shares");
    assertEq(vault.lastTotalAssets(), INITIAL_DEPOSIT, "initial lastTotalAssets");
    assertEq(erc4626Vault.totalAssets(), INITIAL_DEPOSIT, "initial totalAssets");
    assertEq(idleMarket.expectedSupplyAssets(address(vault)), INITIAL_DEPOSIT, "idle
    ↪ market balance");

    _setCap(idleMarket, 0);

    vm.prank(CURATOR);
    vault.submitMarketRemoval(idleMarket);

    vm.warp(block.timestamp + vault.timelock());

    uint256[] memory indexes = new uint256[](0);

    vm.prank(ALLOCATOR);
    vault.updateWithdrawQueue(indexes);

    assertFalse(vault.config(idleMarket).enabled, "market removed");
    assertEq(idleMarket.expectedSupplyAssets(address(vault)), INITIAL_DEPOSIT, "stale
    ↪ market assets remain");
    assertEq(vault.lastTotalAssets(), INITIAL_DEPOSIT, "removal keeps lastTotalAssets
    ↪ inflated");
    assertEq(erc4626Vault.totalAssets(), INITIAL_DEPOSIT, "lostAssets keeps
    ↪ totalAssets flat after removal");

    vm.prank(CURATOR);
    vault.submitCap(idleMarket, type(uint184).max);

    vm.warp(block.timestamp + vault.timelock());
    vault.acceptCap(idleMarket);

    uint256 ghostInflatedTotalAssets = erc4626Vault.totalAssets();
    console2.log("totalAssets after re-enable", ghostInflatedTotalAssets);
    console2.log("lastTotalAssets after re-enable", vault.lastTotalAssets());

    assertEq(ghostInflatedTotalAssets, INITIAL_DEPOSIT * 2, "same market balance
    ↪ counted twice");
    assertEq(vault.lastTotalAssets(), INITIAL_DEPOSIT * 2, "double-counted
    ↪ checkpoint");

    loanToken.setBalance(ONBEHALF, INITIAL_DEPOSIT);

    vm.prank(ONBEHALF);
    uint256 victimShares = erc4626Vault.deposit(INITIAL_DEPOSIT, ONBEHALF);

    console2.log("victim shares minted", victimShares);
    assertApproxEqAbs(victimShares, INITIAL_DEPOSIT / 2, 1, "victim only receives
    ↪ half shares");

    vm.prank(SUPPLIER);
    uint256 attackerAssetsOut = erc4626Vault.redeem(attackerShares, SUPPLIER,
    ↪ SUPPLIER);

```

```

    console2.log("attacker assets redeemed", attackerAssetsOut);
    console2.log("victim max withdraw after attack",
        ↪   erc4626Vault.maxWithdraw(ONBEHALF));

    assertGt(attackerAssetsOut, 190 ether, "attacker drains nearly the entire pool");
    assertLt(erc4626Vault.maxWithdraw(ONBEHALF), 2 ether, "victim is left with dust
        ↪   liquidity");
}
}

```

The proof of concept demonstrates that after removing and re-enabling a market that still holds the vault's 100e18 residual position, the vault reports `totalAssets = 200e18` and `lastTotalAssets = 200e18` even though only 100e18 of real assets exist. A later 100e18 depositor then receives only about 50 shares instead of 100, while the original depositor can redeem against the inflated accounting base and extract nearly the entire pool.

Recommendation: Consider preventing re-enabling of a disabled market while the vault still holds residual supply shares there. This would avoid the need to carry extra bookkeeping through forced-removal flows and would make the market lifecycle consistent with the current accounting model.

If re-enabling residual positions is a desired feature, an alternative would be to track whether the removed position has already been preserved in accounting and only add the live balance delta on re-enable.

Alto Money: Fixed in [PR 491](#).

Phaze: [PR 491](#) addresses the double-counting case by isolating quarantined assets on a per-market basis, effectively mitigating the inflation of `lastTotalAssets` when a force-removed market is re-enabled. Some residual edge cases remain when force-removed markets are re-enabled and the market value has increased in the interim — performance fee accounting on net gains in that path is not included, though this is considered an optional improvement given the rarity of the scenario.

3.3.4 `erc4626Mint()` and `erc4626Withdraw()` do not sweep leftover tokens unlike USM counterparts

Severity: Informational

Context: [AltoAdapter.sol#L218-L241](#)

Description: The `AltoAdapter` contract provides two families of functions for interacting with external protocols: ERC4626 vault functions and USM functions. These families handle leftover tokens inconsistently.

The USM functions `usmBuyAssetWithERC20Transfer()` and `usmSellAssetWithERC20Transfer()` follow a pattern where, after the core operation completes, any tokens not consumed are automatically swept back to the initiator. For example, in `usmBuyAssetWithERC20Transfer()`, if fewer stable tokens are sold than the `maxStableTokenAmount` provided, the difference is transferred back:

```

if (maxStableTokenAmount - stableTokenSold != 0) {
    SafeERC20.safeTransfer(IERC20(stableToken), initiator(), maxStableTokenAmount -
        ↪   stableTokenSold);
}

```

In contrast, `erc4626Mint()` and `erc4626Withdraw()` are "exact output" operations that may consume a variable amount of input tokens, yet they do not include any mechanism to sweep unconsumed tokens from the adapter. After `erc4626Mint()` calls `IERC4626(vault).mint(shares, receiver)`, it resets the approval and performs a slippage check, but any underlying tokens remaining on the adapter are left in place. Similarly, `erc4626Withdraw()` burns a variable number of shares from the owner but does not handle any leftover share tokens.

While the adapter's documentation states that callers are responsible for ensuring no funds remain on the adapter post-transaction, the USM functions already demonstrate a more robust pattern that handles this automatically. Applying the same approach to `erc4626Mint()` and `erc4626Withdraw()` would improve consistency and reduce the risk of tokens being stranded on the adapter due to incomplete bundling.

Recommendation: Consider mirroring the USM sweep pattern in `erc4626Mint()` and `erc4626Withdraw()`. After the core vault operation, the functions could calculate the unconsumed token amount and transfer it back to the receiver. For `erc4626Mint()`, this could look like:

```
uint256 assets = IERC4626(vault).mint(shares, receiver);

SafeERC20.forceApprove(underlyingToken, vault, 0);

require(assets.rDivUp(shares) <= maxSharePriceE27, ErrorsLib.SlippageExceeded());
+
+ uint256 remaining = underlyingToken.balanceOf(address(this));
+ if (remaining != 0) {
+   SafeERC20.safeTransfer(underlyingToken, receiver, remaining);
+ }
```

A similar pattern could be applied to `erc4626Withdraw()` to sweep any leftover share tokens. This would bring the ERC4626 functions in line with the existing USM function behavior and reduce reliance on correct external bundling to prevent token stranding.

Alto Money: Fixed in [PR 495](#).

Phaze: Fix verified.

3.3.5 Current guardian can indefinitely block its own replacement by revoking pending guardian transitions

Severity: Informational

Context: [AltoVault.sol#L463-L468](#)

Description: The guardian transition flow in `AltoVault` follows a submit-timelock-accept pattern: the owner calls `submitGuardian()` to propose a new guardian, a timelock period must elapse, and then `acceptGuardian()` finalizes the transition. During the timelock window, `revokePendingGuardian()` can be called to cancel the pending transition.

The `revokePendingGuardian()` function uses the `onlyGuardianRole` modifier, which permits both the owner and the current guardian to call it:

```
modifier onlyGuardianRole() {
    if (_msgSender() != owner() && _msgSender() != guardian) revert NotGuardianRole();
    _;
}
```

This means the current guardian can revoke any pending guardian submission made by the owner. Since `submitGuardian()` reverts if a `pendingGuardian` is already set, the owner must wait for a revocation to clear before resubmitting. A guardian acting in bad faith could front-run or immediately revoke every new submission, effectively making itself irremovable through the standard transition mechanism.

While the interface documentation includes a warning that “a malicious guardian could disrupt the vault's operation, and would have the power to revoke any pending guardian,” there is no functional reason for the guardian to have access to this function. The owner already has the ability to revoke pending transitions, and only the owner can submit new guardians. Granting the guardian revocation power provides no operational benefit while introducing a mechanism for the guardian to entrench itself.

Recommendation: Consider restricting `revokePendingGuardian()` to `onlyOwner` instead of `onlyGuardianRole`. The owner retains full control over the guardian transition lifecycle, while the current guardian loses the ability to block its own replacement:

```
- function revokePendingGuardian() external onlyGuardianRole {
+ function revokePendingGuardian() external onlyOwner {
    delete pendingGuardian;
```

```
    emit RevokePendingGuardian(_msgSender());  
}
```

Alto Money: Fixed in [PR 496](#).

Phaze: Fix verified.

3.3.6 Lost asset recovery mechanism and its accounting implications should be documented more thoroughly

Severity: Informational

Context: [IAltVault.sol#L244-L245](#)

Description: The `AltVault` tracks permanent asset losses from realized bad debt or forced market removal through a `lostAssets` counter. The interface documents a single recovery method:

```
/// @notice Stores the missing assets due to realized bad debt or forced market removal.  
/// @dev In order to cover those lost assets, it is advised to supply on behalf of  
↪ address(1) on the vault  
/// (canonical method).  
function lostAssets() external view returns (uint256);
```

The `lostAssets` value only climbs upward. When `realTotalAssets` falls below `lastTotalAssets - lostAssets`, the difference is recorded as a new loss. However, `lostAssets` is never automatically reduced when `realTotalAssets` recovers organically through interest accrual on the underlying markets.

This creates a subtle accounting dynamic that is not documented. Since `totalAssets()` is computed as `realTotalAssets + lostAssets`, any organic growth in `realTotalAssets` from market interest increases `totalAssets` and is treated as new yield. Fees are assessed on this yield even though the vault's real asset base has not yet recovered to its pre-loss level. In effect, the interest that gradually restores real liquidity becomes fee-bearing before the impairment has been made whole.

The canonical recovery path, donating to `address(1)`, avoids this by directly increasing `realTotalAssets` such that on the next accrual, `lostAssets` can decrease. However, this creates a permanent "ghost position" at `address(1)` that compounds with all future interest, acting as a form of built-in hedge against future losses but also diluting other shareholders indefinitely.

Neither of these behavioral nuances, the fee implications of organic recovery nor the long-term effects of the `address(1)` ghost position, are documented in the contract or its interface.

Recommendation: Consider expanding the documentation around the `lostAssets` recovery mechanism to cover:

1. That `lostAssets` does not decrease automatically, even as underlying market interest restores real liquidity.
2. That organic recovery through interest is treated as new fee-bearing yield, making donation to `address(1)` the preferred path to avoid fee double-counting on the recovery amount.
3. That shares held by `address(1)` compound perpetually with future interest, and the implications this carries for existing shareholders.

Alto Money: Fixed in [PR 498](#).

Phaze: Fix verified.